

01 SaborExpress Especificacion de Requisitos SRS

SABOREXPRESS — PROYECTO DE SOFTWARE

Yerson Stiven Cuellar Rubiano

Ficha de formación: 3145555

Expediente técnico y académico — APA 7

14 de octubre de 2026

Fuente documental: repositorios SaborExpressBack, SaborExpressWeb y SaborExpressMovil.

Las pruebas no ejecutadas se identifican como pendientes.

Especificación de requisitos de software (SRS)

Proyecto: SaborExpress | **Autor / integrante:** Yerson Stiven Cuellar Rubiano | **Ficha:** 3145555 | **Fecha:** 14 de octubre de 2026 | **Versión documental:** 4.0

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Propósito, alcance y lectores

SaborExpress es una solución de apoyo a operaciones gastronómicas con API central, portal administrativo web y aplicación móvil. Este documento traduce capacidades reconocibles del repositorio a requisitos verificables; no constituye aceptación contractual ni certificación del sistema. Destinatarios: equipo de desarrollo, analista, instructor, QA, operación y responsables de producto.

2. Contexto y límites

Comprende registro/autenticación, gestión de sedes, personal, productos, pedidos, pagos, repartos, notificaciones y administración. Integra Wompi mediante endpoints de inicialización, conciliación y webhook; depende de red y credenciales de servicios externos. No se confirma en estos repositorios una aplicación de escritorio independiente, despliegue productivo estable ni certificación bancaria.

Actores

Gerente, administrador de sede, cliente, cajero, cocinero, mesero y repartidor. Los nombres exactos de roles y las asignaciones de permisos deben confrontarse con `Shared/Constants/RoleNames` y los seeders. El proveedor Wompi participa como sistema externo.

3. Requisitos funcionales

Convención: la columna «evidencia» significa que el código contiene un punto de entrada o pantalla; la columna no afirma que el caso de uso pasó una prueba funcional.

ID	Actor	Comportamiento requerido	Evidencia inicial	Prioridad	Verificación
RF-01	Usuario	Iniciar sesión con credenciales	POST /api/Auth/login	Alta	API + cliente de login
RF-02	Usuario	Recuperar contraseña mediante código y restablecimiento	AuthController	Alta	API
RF-03	Administrador	Administrar permisos y asignación de roles	Controllers/ Roles,Permissions, UserRoles	Alta	API
RF-04	Gerente	Registrar y actualizar sedes	BranchesController	Alta	API
RF-05	Administrador	Consultar su sede asignada	GET /api/Branches/mine	Alta	API
RF-06	Administrador	Gestionar empleados	EmployeesController	Alta	API
RF-07	Personal	Consultar horarios y turnos	EmployeeSchedulesController	Media	API y cliente
RF-08	Administrador	Gestionar categorías y productos	ProductsController/ CategoriesController	Alta	API
RF-09	Cliente	Ver productos por sede o categoría	GET /api/Products	Alta	API
RF-10	Cliente/cajero	Crear pedidos con detalle de productos	POST /api/Orders	Alta	API
RF-11	Personal	Filtrar pedidos por mesa, cliente y sede	GET /api/Orders/…	Alta	API
RF-12	Personal	Actualizar estado de pedido con autorización	PATCH /api/Orders/{id}/status	Alta	API
RF-13	Cliente	Cancelar pedido propio según restricciones	PATCH /api/Orders/{id}/cancel-mine	Alta	API
RF-14	Cajero	Registrar y consultar pagos por turno	PaymentsController	Alta	API
RF-15	Cliente	Iniciar pago digital Wompi	POST /api/Payments/wompi/init	Alta	API
RF-16	Proveedor de pago	Informar resultado por webhook autenticado	POST /api/Payments/wompi/webhook	Crítica	API; validar firma
RF-17	Cajero	Iniciar cobro con QR Wompi	POST /api/Payments/wompi/cashier-init	Media	API
RF-18	Gerente	Asignar repartidor a entrega	POST /api/Deliveries	Alta	API
RF-19	Repartidor	Actualizar estado de entrega autorizado	PATCH /api/Deliveries/{id}/status	Alta	API
RF-20	Cliente	Calificar reparto asociado	POST /api/Deliveries/{id}/rating	Media	API
RF-21	Usuario	Consultar sus notificaciones	NotificationsController	Media	API y cliente
RF-22	Cocina	Visualizar tablero	PublicKitchenCont	Alta	API y móvil

ID	Actor	Comportamiento requerido	Evidencia inicial	Prioridad	Verificación
		de pedidos en preparación	roller y componente KitchenBoardPublic		
RF-23	Cliente	Consultar sede más cercana por coordenadas	GET /api/Branches/nearest	Media	API
RF-24	Personal	Gestionar mesas, pedidos y menú diario	Tables,Orders,DailyMenu Controllers	Alta	API
RF-25	Administrador	Consultar reportes operativos web	src/app/reports/page.tsx	Media	Página identificada; funcionalidad a probar

4. Requisitos no funcionales

Los criterios siguientes son objetivos de aceptación propuestos, no mediciones efectuadas.

ID	Calidad	Criterio	Cómo verificar
RNF-01	Seguridad	Autorización por rol y por recurso (sede, propietario) en cada operación sensible	0 accesos cruzados en batería negativa
RNF-02	Autenticación	JWT con expiración, validación, rotación o revocación según diseño	Pruebas token vencido/manipulado
RNF-03	Pagos	Webhook con validación criptográfica, idempotencia y rastreo de referencia	Reenvío idéntico no duplica abono
RNF-04	Integridad	Transacciones coherentes para pedido/pago/entrega	Sin estados contradictorios en escenarios de error
RNF-05	Rendimiento	Definir carga nominal, p50/p95 HTTP y SignalR	Medir con entorno versionado; sin umbral inventado
RNF-06	Disponibilidad	Recuperación de caídas de API y base de datos	Ensayos de fallo y recuperación
RNF-07	Portabilidad	Instalación reproducible en entorno de desarrollo y Docker	Documentar variables y dependencias
RNF-08	Mantenibilidad	Organización modular controller/service/repository	Revisión por pares + checks estáticos
RNF-09	Compatibilidad	Ensayar proveedor relacional realmente elegido	Migraciones y CRUD pasan en motor escogido
RNF-10	Usabilidad	Tareas críticas de cliente, cajero y repartidor	Prueba con participantes y tasa de éxito
RNF-11	Accesibilidad	Etiquetas, navegación por teclado y contraste adecuados	Auditoría automática + revisión manual
RNF-12	Privacidad	Minimización de datos y no exposición de claves	Escaneo de secretos + políticas de acceso
RNF-13	Observabilidad	Errores trazables sin credenciales ni datos financieros	Correlación de logs por transacción
RNF-14	Internacionalización	Mensajes consistentes con preferencias del usuario	Pruebas idioma/locale
RNF-15	Respaldo	Copias y restauración comprobadas del proveedor seleccionado	Ensayo aislado documentado

5. Reglas de negocio y restricciones

1. Un pedido se vincula a una sede; puede provenir de cliente, empleado o mesa y opcionalmente asociarse a una dirección de entrega.
2. Order.CustomerId, EmployeeId, TableId y AddressId permiten escenarios distintos. El servicio debe validar la coherencia según tipo de pedido.
3. Un detalle identifica producto, cantidad, precio unitario, subtotal y estado; la edición del detalle deja trazabilidad histórica cuando corresponda.
4. Un pago pertenece a un pedido. CashierId puede ser nulo en la integración digital directa.
5. El controlador de pedidos contempla una restricción para no confirmar manualmente pedidos con intentos recientes de Wompi pendientes. El comportamiento completo exige prueba de servicio y webhook.
6. El acceso de empleados y repartidores depende de roles, sede y en algunas acciones de la política ActiveShift.
7. Productos con BranchId nulo se interpretan en el modelo como globales; los asignados a sede se restringen a esa sede.
8. El flujo de cambios sensibles debe considerar auditoría, permisos y validación de propiedad del recurso.

6. Interfaces y dependencias

Cliente web: Next.js 16, React 19. Cliente móvil: Expo 57, React Native 0.86. API: ASP.NET Core 8 con JSON, JWT Bearer y SignalR. Persistencia: EF Core y motor seleccionable MySQL, SQL Server o PostgreSQL. Servicios externos: Wompi, correo y geocodificación donde el despliegue los configure.

7. Escenarios de aceptación por flujo

Pedido de cliente: autenticarse, consultar catálogo por sede, seleccionar productos, confirmar tipo/dirección, registrar pedido, revisar importe y consultar estado. Resultado esperado: un solo pedido, suma calculada correctamente, vínculo a sede y autorización del cliente.

Pago digital: iniciar transacción con referencia única, procesar estado en sandbox, aceptar webhooks válidos, rechazar firma inválida, tolerar eventos duplicados. Resultado esperado: sin doble acreditación ni confirmación anticipada.

Repartidor: consultar solo entregas permitidas, actualizar estado, impedir modificar reparto de otra sede, permitir calificación de cliente autorizado.

8. Control de cambios y aceptación

Toda historia debe registrar identificador, versión, fuente, responsable, dependencias, prueba positiva y negativa, evidencia anonimizada, resultado y aprobación. Documentos de diseño y endpoints derivan de estos requisitos mediante la matriz de trazabilidad incluida en 9-traceability.

9. Fichas completas de requisitos funcionales

Se describen los requisitos recuperados del código fuente con una especificación verificable de comportamiento. La presencia de una ruta HTTP es evidencia del punto de entrada, no aceptación del requisito. Los criterios explícitos se proponen para formalizar la revisión del sistema conforme a ISO/IEC/IEEE 29148:2018.

RF-01. Iniciar sesión con credenciales

Actor responsable: Usuario. **Prioridad propuesta:** Alta. **Rastreo al código:** POST /api/Auth/login. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: correo y contraseña sintéticos en ambiente de prueba. **Disparador:** credenciales correctas y cuenta habilitada.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** devolver respuesta autenticada y restringir rutas según roles.

Alternativas y errores: usuario inexistente, clave incorrecta, cuenta bloqueada. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-01, CP-02, CP-03. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-02. Recuperar contraseña mediante código y restablecimiento

Actor responsable: Usuario. **Prioridad propuesta:** Alta. **Rastreo al código:** AuthController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: cuenta de prueba con correo verificable. **Disparador:** solicitud de código y secuencia de restablecimiento.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** restablecer acceso sin divulgar si la cuenta existe.

Alternativas y errores: código vencido, reuso y exceso de intentos. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-02 y adicional CP-28. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-03. Administrar permisos y asignación de roles

Actor responsable: Administrador. **Prioridad propuesta:** Alta. **Rastreo al código:** Controllers/Roles,Permissions,UserRoles. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: administrador autorizado y usuario de prueba. **Disparador:** consultar roles, asociar usuario, revocar permisos.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** la siguiente solicitud observa las autorizaciones resultantes.

Alternativas y errores: usuario sin rol intenta elevar privilegios. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-05 y adicional CP-29. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-04. Registrar y actualizar sedes

Actor responsable: Gerente. **Prioridad propuesta:** Alta. **Rastreo al código:** BranchesController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: sede ficticia con nombre, dirección y estado. **Disparador:** dar de alta sede y editar datos permitidos.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** persistir cambios e impedir acceso no autorizado.

Alternativas y errores: ID inexistente y edición por empleado. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-05. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-05. Consultar su sede asignada

Actor responsable: Administrador. **Prioridad propuesta:** Alta. **Rastreo al código:** GET /api/Branches/mine. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: dos sedes ficticias con administradores separados. **Disparador:** autenticar cada administrador y consultar mine.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** devolver solo sede autorizada.

Alternativas y errores: intento de enumeración de sedes ajenas. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-05. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-06. Gestionar empleados

Actor responsable: Administrador. **Prioridad propuesta:** Alta. **Rastreo al código:** EmployeesController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: empleado ficticio y rol de administración. **Disparador:** crear, actualizar y consultar empleado de la propia sede.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** proteger datos personales y autorización por sede.

Alternativas y errores: documento duplicado y edición intersede. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-05. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-07. Consultar horarios y turnos

Actor responsable: Personal. **Prioridad propuesta:** Media. **Rastreo al código:** EmployeeSchedulesController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: turno y horario de prueba configurados. **Disparador:** consultar agenda y solicitar operación con turno vigente.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** validar vigencia cuando se requiera política ActiveShift.

Alternativas y errores: turno finalizado o sin asignación. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-30. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-08. Gestionar categorías y productos

Actor responsable: Administrador. **Prioridad propuesta:** Alta. **Rastreo al código:** ProductsController/CategoriesController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: categoría y producto con precio positivo. **Disparador:** registrar artículo con categoría y sede opcional.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** producto recuperable desde los listados autorizados.

Alternativas y errores: producto sin categoría o modificación sin rol. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-06, CP-07. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-09. Ver productos por sede o categoría

Actor responsable: Cliente. **Prioridad propuesta:** Alta. **Rastreo al código:** GET /api/Products. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: productos globales y exclusivos de sedes A/B. **Disparador:** obtener GET /api/Products?branchId=A.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta

persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** catálogo de A más artículos globales.

Alternativas y errores: pedido de sede inexistente. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-06. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-10. Crear pedidos con detalle de productos

Actor responsable: Cliente/cajero. **Prioridad propuesta:** Alta. **Rastreo al código:** POST /api/Orders. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: carrito sintético de cantidades/precios. **Disparador:** crear orden con sede, canal y detalle.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** guardar total consistente, detalles y trazabilidad de estado.

Alternativas y errores: cantidad inválida y domicilio sin dirección. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-08, CP-09, CP-10. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-11. Filtrar pedidos por mesa, cliente y sede

Actor responsable: Personal. **Prioridad propuesta:** Alta. **Rastreo al código:** GET /api/Orders/.... **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedidos de dos clientes y sedes. **Disparador:** filtrar listados de pedidos por parámetros.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** respuesta paginada/filtrada según contrato real.

Alternativas y errores: consulta ajena o filtros incompatibles. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-05, CP-12. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-12. Actualizar estado de pedido con autorización

Actor responsable: Personal. **Prioridad propuesta:** Alta. **Rastreo al código:** PATCH /api/Orders/{id}/status. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedido pendiente y política de transiciones. **Disparador:** solicitar PATCH status con empleado autorizado.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** conservar historial y evitar saltos prohibidos.

Alternativas y errores: estado final irreversible y Wompi pendiente. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-13. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-13. Cancelar pedido propio según restricciones

Actor responsable: Cliente. **Prioridad propuesta:** Alta. **Rastreo al código:** PATCH /api/Orders/{id}/cancel-mine. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedido asociado a cliente ficticio. **Disparador:** el mismo cliente solicita cancelación antes de cocina.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** cambiar estado bajo reglas existentes.

Alternativas y errores: cliente distinto o pedido ya preparado. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-11, CP-12. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-14. Registrar y consultar pagos por turno

Actor responsable: Cajero. **Prioridad propuesta:** Alta. **Rastreo al código:** PaymentsController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: cajero con turno activo y pagos de otros cajeros. **Disparador:** registrar cobro y consultar my-shift.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta

persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** mostrar solo los importes pertinentes al turno.

Alternativas y errores: importe negativo y mezcla de cajeros. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-19. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-15. Iniciar pago digital Wompi

Actor responsable: Cliente. **Prioridad propuesta:** Alta. **Rastreo al código:** POST /api/Payments/wompi/init. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedido válido pendiente y Wompi sandbox. **Disparador:** solicitar init de pago.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** generar referencia y monto asociados al pedido.

Alternativas y errores: orden ajena o total manipulado. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-14, CP-17. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-16. Informar resultado por webhook autenticado

Actor responsable: Proveedor de pago. **Prioridad propuesta:** Crítica. **Rastreo al código:** POST /api/Payments/wompi/webhook. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: evento sandbox con checksum y referencia. **Disparador:** enviar webhook legítimo y duplicarlo.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** autenticidad validada e idempotencia conservada.

Alternativas y errores: firma errónea, referencia inexistente. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-14, CP-15, CP-16. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-17. Iniciar cobro con QR Wompi

Actor responsable: Cajero. **Prioridad propuesta:** Media. **Rastreo al código:** POST /api/Payments/wompi/cashier-init. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: cajero autorizado, pedido y medio Wompi. **Disparador:** iniciar cobro mediante cashier-init.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** vincular QR/referencia al pedido sin duplicación.

Alternativas y errores: turno inválido y orden previamente pagada. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-18. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-18. Asignar repartidor a entrega

Actor responsable: Gerente. **Prioridad propuesta:** Alta. **Rastreo al código:** POST /api/Deliveries. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedido domicilio y repartidor activo en sede. **Disparador:** asignar entrega desde un gerente autorizado.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** persistir orden, dirección y repartidor correctos.

Alternativas y errores: repartidor de otra sede o inactivo. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-20. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-19. Actualizar estado de entrega autorizado

Actor responsable: Repartidor. **Prioridad propuesta:** Alta. **Rastreo al código:** PATCH /api/Deliveries/{id}/status. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: reparto asignado al empleado A. **Disparador:** modificar estado como repartidor A.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** solo transición autorizada y registro actualizado.

Alternativas y errores: repartidor B intenta modificar. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-20. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-20. Calificar reparto asociado

Actor responsable: Cliente. **Prioridad propuesta:** Media. **Rastreo al código:** POST /api/Deliveries/{id}/rating. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: entrega finalizada del cliente titular. **Disparador:** calificar 1 a 5 y comentario de prueba.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** almacenar evaluación asociada a entrega.

Alternativas y errores: valor fuera de rango o tercero. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-21. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-21. Consultar sus notificaciones

Actor responsable: Usuario. **Prioridad propuesta:** Media. **Rastreo al código:** NotificationsController. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: usuario con notificaciones propias/ajenas. **Disparador:** consultar notificaciones del usuario autenticado.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** evitar exponer mensajes y tokens ajenos.

Alternativas y errores: solicitud de ID de otro usuario. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-31. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-22. Visualizar tablero de pedidos en preparación

Actor responsable: Cocina. **Prioridad propuesta:** Alta. **Rastreo al código:** PublicKitchenController y componente KitchenBoardPublic. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: pedidos con etapas preparadas en sede de prueba. **Disparador:** abrir tablero de cocina y actualizar pedido.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta

persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** estados visibles/reconciliables tras reconexión.

Alternativas y errores: dos operadores simultáneos y red intermitente. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-22. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-23. Consultar sede más cercana por coordenadas

Actor responsable: Cliente. **Prioridad propuesta:** Media. **Rastreo al código:** GET /api/Branches/nearest. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: sedes con coordenadas sintéticas. **Disparador:** consultar nearest con latitud/longitud admisibles.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** retornar sede elegible según política calculada.

Alternativas y errores: coordenadas nulas, fuera de rango. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-32. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-24. Gestionar mesas, pedidos y menú diario

Actor responsable: Personal. **Prioridad propuesta:** Alta. **Rastreo al código:** Tables,Orders,DailyMenu Controllers. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: mesa, orden y menú configurado. **Disparador:** registrar flujo operativo de mesa/menú.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** asociar sede, mesa y detalle sin duplicidad.

Alternativas y errores: mesa ocupada y cambios simultáneos. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-33. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

RF-25. Consultar reportes operativos web

Actor responsable: Administrador. **Prioridad propuesta:** Media. **Rastreo al código:** src/app/reports/page.tsx. **Dependencias:** identidad autorizada, sede y datos consistentes según el dominio.

Precondiciones y datos: datos sintéticos por dos sedes. **Disparador:** abrir página web de reportes como gerente.

Flujo principal verificable: 1) la interfaz recopila datos y valida su forma; 2) el cliente invoca el contrato HTTP bajo el rol adecuado; 3) la capa de servicio aplica las reglas y consulta persistencia; 4) el servidor devuelve resultado tipado y la interfaz actualiza su estado. **Criterio de aceptación:** visualización consistente con permisos y origen de datos.

Alternativas y errores: rol no autorizado y rango sin datos. Debe registrarse estado inicial y final, mensaje sin filtración de datos, código HTTP esperado según implementación, y comportamiento ante reintento.

Validación asociada: CP-34. **Estado de cumplimiento:** por confirmar mediante pruebas ejecutadas en la rama y SHA entregados.

10. Especificación operacional de requisitos no funcionales

Los atributos de calidad no pueden deducirse únicamente de la presencia de librerías. Para cada RNF se consigna el mecanismo de evaluación, las evidencias exigidas y el criterio para determinar aprobación; cuando no hay objetivo numérico contractual el umbral debe fijarse antes de la ejecución.

RNF-01. Seguridad

Intención de calidad: Autorización por rol y por recurso (sede, propietario) en cada operación sensible.

Método de verificación: 0 accesos cruzados en batería negativa. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-02. Autenticación

Intención de calidad: JWT con expiración, validación, rotación o revocación según diseño.

Método de verificación: Pruebas token vencido/manipulado. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-03. Pagos

Intención de calidad: Webhook con validación criptográfica, idempotencia y rastreo de referencia.

Método de verificación: Reenvío idéntico no duplica abono. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-04. Integridad

Intención de calidad: Transacciones coherentes para pedido/pago/entrega.

Método de verificación: Sin estados contradictorios en escenarios de error. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-05. Rendimiento

Intención de calidad: Definir carga nominal, p50/p95 HTTP y SignalR.

Método de verificación: Medir con entorno versionado; sin umbral inventado. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-06. Disponibilidad

Intención de calidad: Recuperación de caídas de API y base de datos.

Método de verificación: Ensayos de fallo y recuperación. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-07. Portabilidad

Intención de calidad: Instalación reproducible en entorno de desarrollo y Docker.

Método de verificación: Documentar variables y dependencias. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-08. Mantenibilidad

Intención de calidad: Organización modular controller/service/repository.

Método de verificación: Revisión por pares + checks estáticos. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-09. Compatibilidad

Intención de calidad: Ensayar proveedor relacional realmente elegido.

Método de verificación: Migraciones y CRUD pasan en motor escogido. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-10. Usabilidad

Intención de calidad: Tareas críticas de cliente, cajero y repartidor.

Método de verificación: Prueba con participantes y tasa de éxito. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no

aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-11. Accesibilidad

Intención de calidad: Etiquetas, navegación por teclado y contraste adecuados.

Método de verificación: Auditoría automática + revisión manual. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-12. Privacidad

Intención de calidad: Minimización de datos y no exposición de claves.

Método de verificación: Escaneo de secretos + políticas de acceso. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-13. Observabilidad

Intención de calidad: Errores trazables sin credenciales ni datos financieros.

Método de verificación: Correlación de logs por transacción. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-14. Internacionalización

Intención de calidad: Mensajes consistentes con preferencias del usuario.

Método de verificación: Pruebas idioma/locale. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

RNF-15. Respaldo

Intención de calidad: Copias y restauración comprobadas del proveedor seleccionado.

Método de verificación: Ensayo aislado documentado. Seleccionar ambiente, tipo de cliente, perfil autorizado, carga nominal y conjunto de datos sintéticos; guardar SHA, fecha, resultado esperado, resultado observado y evidencia.

Evidencia aceptable: informe de prueba reproducible, archivos de configuración anonimizados y resultado de la revisión por dos personas o el responsable designado. **Criterio de decisión:** no aprobado mientras no se registre ejecución; cualquier falla crítica impide marcar el requisito como validado.

11. Matriz de trazabilidad inicial

ID	API/componente	Prueba trazada	Responsable de aceptación
RF-01	POST /api/Auth/login	CP-01, CP-02, CP-03	Por designar
RF-02	AuthController	CP-02 y adicional CP-28	Por designar
RF-03	Controllers/	CP-05 y adicional CP-29	Por designar

ID	API/componente	Prueba trazada	Responsable de aceptación
	Roles,Permissions,UserRoles		
RF-04	BranchesController	CP-05	Por designar
RF-05	GET /api/Branches/mine	CP-05	Por designar
RF-06	EmployeesController	CP-05	Por designar
RF-07	EmployeeSchedulesController	CP-30	Por designar
RF-08	ProductsController/ CategoriesController	CP-06, CP-07	Por designar
RF-09	GET /api/Products	CP-06	Por designar
RF-10	POST /api/Orders	CP-08, CP-09, CP-10	Por designar
RF-11	GET /api/Orders/…	CP-05, CP-12	Por designar
RF-12	PATCH /api/Orders/{id}/status	CP-13	Por designar
RF-13	PATCH /api/Orders/{id}/cancel-mine	CP-11, CP-12	Por designar
RF-14	PaymentsController	CP-19	Por designar
RF-15	POST /api/Payments/wompi/init	CP-14, CP-17	Por designar
RF-16	POST /api/Payments/wompi/webhook	CP-14, CP-15, CP-16	Por designar
RF-17	POST /api/Payments/wompi/cashier-init	CP-18	Por designar
RF-18	POST /api/Deliveries	CP-20	Por designar
RF-19	PATCH /api/Deliveries/{id}/status	CP-20	Por designar
RF-20	POST /api/Deliveries/{id}/rating	CP-21	Por designar
RF-21	NotificationsController	CP-31	Por designar
RF-22	PublicKitchenController y componente KitchenBoardPublic	CP-22	Por designar
RF-23	GET /api/Branches/nearest	CP-32	Por designar
RF-24	Tables,Orders,DailyMenu Controllers	CP-33	Por designar
RF-25	src/app/reports/page.tsx	CP-34	Por designar

12. Control de líneas base y solicitud de cambio

La línea base debe identificar hash de cada repositorio, versión de esta SRS, fecha de revisión, aprobación del representante de negocio y estado individual de requisitos. Toda solicitud de cambio (CR) indicará motivo, necesidad del usuario, módulos afectados, riesgo de regresión, pruebas que hay que repetir y estimación. Los requisitos se identifican de forma estable; no reutilizar identificadores dados de baja. Un requisito se considera aceptado solo si su prueba de aceptación tiene evidencia, existe revisión y la documentación de usuario se actualiza cuando corresponde.

Campos del registro CR: ID, solicitante, fecha, descripción actual, nuevo comportamiento, prioridad, impacto en API/BD/cliente, cambio de datos, riesgos de seguridad, evaluador, decisión, commit, fecha de despliegue y rollback.

Referentes: ISO/IEC/IEEE 29148:2018 y ISO/IEC 25010:2023 (modelos normativos, no certificación del proyecto).

Fuentes primarias de implementación

- API: https://github.com/yersonct/SaborExpressBack/tree/main
- Web: https://github.com/yersonct/SaborExpressWeb/tree/main
- Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main
- Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.

13. Restricciones externas, dependencias y atributos verificables

La aplicación opera con un backend central de ASP.NET Core 8. Las interfaces deben recibir y mostrar el estado real provisto por la API; los clientes no deben asignar importes finales ni autorizar operaciones mediante estados locales. Para una sede, `BranchId` define ámbito operativo y requiere prueba de autorización a nivel de objeto. Un rol válido por sí solo no habilita acceder a pedidos de terceros.

Interfaces: JSON sobre HTTP; autenticación bearer para áreas protegidas; canal SignalR identificado en `/hubs/notifications`; proveedor de pagos Wompi; persistencia EF Core; aplicación Next.js y Expo. Si cambian rutas, DTOs o estados, se actualiza simultáneamente OpenAPI, manual técnico, clientes y matriz RF ↔ prueba.

14. Matriz de operaciones / roles para verificar

Operación	Cliente	Cajero	Cocinero	Repartidor	Gerente / administrador	Consideraciones
Ver catálogo público	Permitido según ruta pública	Sí	Sí	Sí	Sí	GET /api/Products admite anónimo
Crear un pedido	Propio	Según turno	Según autorización	No previsto	Según permisos	OrdersController aplica JWT y política ActiveShift
Cancelar pedido propio	Sí con restricciones	No por la ruta cancel-mine	No	No	Gestión separada	Requiere prueba de propiedad y estado
Registrar cobro	No por operación de caja	Sí según backend	No	No	Según permisos	Validar su rol exacto en servicio
Calificar entrega	Solo asociado al reparto	No	No	No	No	Endpoint marcado Cliente
Crear producto	No	No	No	No	Gerente y Administrador	Restricción declarativa en controlador
Crear sede	No	No	No	No	Gerente	BranchesController lo limita a Gerente

Nota. Esta tabla es una **matriz de verificación**, no una certificación de autorización. Algunos roles deben contrastarse con RoleNames y pruebas de integración.

15. Diagramas de alcance funcional

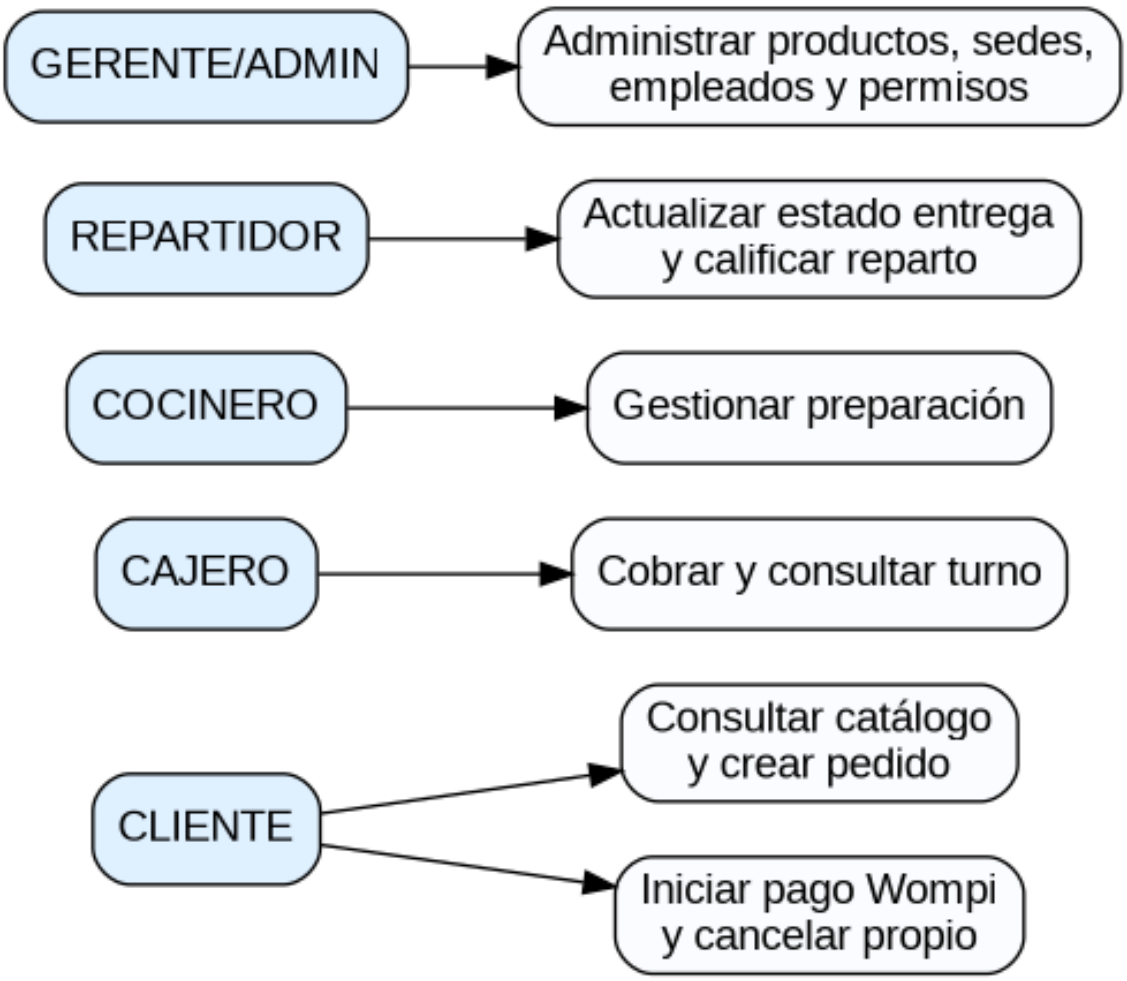


Figura 8. Casos de uso por actor

08_CASOS_USO

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas.

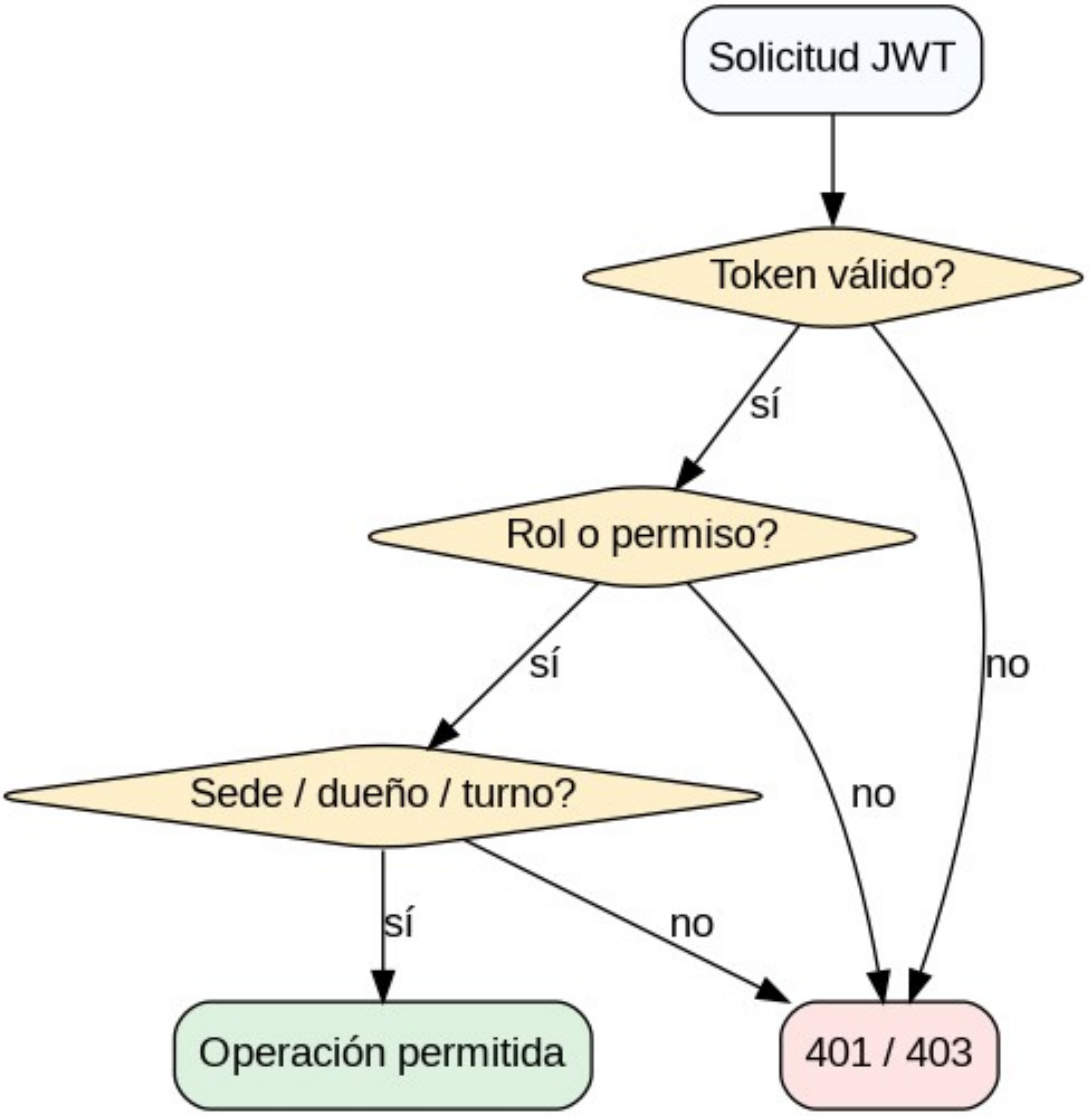


Figura 9. Controles de autorización que deben validarse

09_AISLAMIENTO_ROLES

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas. ## 16. Plantilla de aceptación por requisito

Cada RF contiene identificador único, definición inequívoca, actor, precondición, disparador, flujo alterno, regla de negocio, dato protegido, dependencia, origen de evidencia, prueba positiva, prueba negativa, prioridad y estado real de validación. Cada RNF requiere atributo

medible, ambiente, herramienta, conjunto de datos y umbral que apruebe el responsable. Los criterios que todavía no tienen umbrales acordados se declaran **por definir**, nunca se reemplazan por números arbitrarios.